

Bash Scripts And Text Analysis

Today we combine small Linux tools to search, filter, count, and automate a ticket report.

Core skills

- Bash scripts
- pipes
- redirection
- grep
- find
- cut, sort, uniq, wc

Industry connection

- logs
- CSV exports
- test output
- security events
- support tickets
- quick evidence reports

What We Already Know

These commands are the base for today's analysis.

```
$ pwd
```

```
$ ls -la
```

```
$ cd ~/bootcamp
```

```
$ cat notes.txt
```

```
$ python3 hello.py
```

If students are lost, return to pwd and ls.

A Lot Of Technical Evidence Is Plain Text

Linux is strong at turning text into answers quickly.

Examples

- application logs
- server logs
- CSV files
- error reports
- command output
- configuration files

Questions

- what failed?
- how many times?
- which category?
- which files match?
- what changed?
- what should I report?

The Pipeline Mental Model

One command's output can become the next command's input.

tickets.csv

- > tail removes the header
- > cut selects one column
- > sort groups matching values
- > uniq -c counts repeated values
- > sort -nr puts largest first

This is the pipe idea without the syntax.

Never Analyze A File Blind

Look at the top, count the lines, and understand the columns.

```
$ head tickets.csv
```

```
$ wc -l tickets.csv
```

```
$ tail tickets.csv
```

The header row is not a ticket record.

Search For Matching Lines

grep is one of the most useful Linux commands for logs and text files.

```
$ grep "network" tickets.csv
```

```
$ grep -i "password" tickets.csv
```

```
$ grep ",high," tickets.csv
```

grep -i ignores case.

Count Tickets By Category

Build the pipeline one step at a time.

```
$ tail -n +2 tickets.csv | cut -d, -f3
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort | uniq -c
```

```
$ tail -n +2 tickets.csv | cut -d, -f3 | sort | uniq -c | sort -nr
```

Run each stage separately when students are confused.

Save Output As Evidence

Technical work often requires a report, not just a command on screen.

```
$ ./ticket_report.sh > report.txt  
$ cat report.txt  
$ ./ticket_report.sh >> report.txt
```

> overwrites. >> appends.

Save The Workflow

A script makes the analysis repeatable.

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
DATA_FILE="${1:-tickets.csv}"
```

```
echo "Tickets by category:"
```

```
tail -n +2 "$DATA_FILE" | cut -d, -f3 | sort | uniq -c | sort -nr
```

Scripts are repeatable command recipes.

Locate Files By Pattern

find searches through folders for matching files.

```
$ find . -name "*.sh"
```

```
$ find . -name "*.txt"
```

```
$ find ~/bootcamp -name "*.py"
```

The dot means start in the current folder.

Commands Are Not The Final Answer

Students need to translate output into a sentence.

Output

- 3 network
- 2 software
- 2 password
- 2 access
- 1 hardware

Explanation

- network is the largest category
- there are 10 ticket records
- four tickets are high priority
- the report is evidence

Pipeline Debugging

Fix one piece at a time.

Typical issues

- file not found
- wrong folder
- missing quotes
- wrong field number
- spaces in copied command
- script not executable

Troubleshoot

- pwd
- ls
- head tickets.csv
- run one command stage
- use bash script.sh
- then try ./script.sh

If We Need The Full Two Hours

This is the slow, comfortable pacing plan.

- 0-10 Course reset and today goal
- 10-20 Review terminal basics
- 20-35 Mini lecture: Bash and text files
- 35-50 Demo: head, wc, grep
- 50-65 Student work block 1
- 65-80 Demo: pipeline one stage at a time
- 80-95 Student work block 2
- 95-110 Demo: ticket_report.sh
- 110-120 Stretch, reflection, exit question

Use this slide if you feel rushed or need to re-center the room.

Five Commands Before We Analyze Anything

This gets everyone oriented in the same folder.

```
$ pwd
```

```
$ ls
```

```
$ ls data
```

```
$ head data/tickets.csv
```

```
$ wc -l data/tickets.csv
```

If students cannot do this, pause before pipelines.

Know The Columns

The commands only make sense if students know what each field means.

- Field 1: id
- Field 2: student_name
- Field 3: category
- Field 4: priority
- Field 5: status
- Field 6: created_date
- Field 7: issue

Today's category count uses field 3.

What Does tail -n +2 Mean?

This command is not about the last two lines.

Common confusion

- tail usually means end of file
- -n normally controls line count
- +2 means start at line 2
- line 1 is the CSV header

Result

- header is skipped
- ticket records remain
- counts are not polluted
- analysis is cleaner

What Does cut -d, -f3 Mean?

cut selects a field from each line.

```
$ cut -d, -f3
```

-d, use comma as the delimiter

-f3 print field 3

In tickets.csv:

field 3 = category

Change -f3 to -f4 for priority or -f5 for status.

Why sort Before uniq -c?

uniq only counts repeated lines that are next to each other.

Without sort:

```
network  
password  
network
```

With sort:

```
network  
network  
password
```

Then uniq -c can count correctly.

This is one of the most important pipeline concepts today.

Two Symbols Students Mix Up

Pipe sends to another command. Redirect sends to a file.

Pipe

```
$ command1 | command2
```

output goes into
another command

Redirect

```
$ command > report.txt
```

output goes into
a file

Pause And Predict

Ask students before running the command.

```
$ tail -n +2 data/tickets.csv | cut -d, -f4 | sort | uniq -c | sort -nr
```

Question:

What are we counting now?

Answer:

Priority, because field 4 is priority.

Use prediction to slow the room down productively.

ticket_report.sh Is Just Saved Commands

Every line has a purpose.

Top

- shebang chooses Bash
- strict mode catches mistakes
- DATA_FILE stores the CSV path

Body

- echo prints labels
- pipelines calculate counts
- grep lists high tickets
- redirection saves report

What Is `set -euo pipefail`?

This line makes the script less forgiving, which is useful for catching mistakes.

```
set -euo pipefail
```

`-e` stop on errors

`-u` stop on unset variables

`pipefail` notice failures inside pipelines

You can say: this is a professional habit, not something to memorize today.

Why Does The Script Use `|| true`?

grep reports failure when it finds no matches.

```
grep ",high," "$DATA_FILE" || true
```

Meaning:

Run grep.

If grep finds nothing, do not crash the whole script.

This is useful when a report section may have zero results.

Add Tickets By Status

Use this if students finish the main lab early.

```
echo "Tickets by status:"  
tail -n +2 "$DATA_FILE" | cut -d, -f5 | sort | uniq -c | sort -nr  
echo
```

Expected result: 7 open, 3 closed.

Save Three Small Reports

This reinforces redirection and file inspection.

```
$ tail -n +2 data/tickets.csv | cut -d, -f3 | sort | uniq -c | sort -nr > categories.txt
$ tail -n +2 data/tickets.csv | cut -d, -f4 | sort | uniq -c | sort -nr > priorities.txt
$ tail -n +2 data/tickets.csv | cut -d, -f5 | sort | uniq -c | sort -nr > statuses.txt
$ ls -l *.txt
```

Use this only if the room is moving quickly.

Explain The Pipeline In Plain English

This turns command copying into understanding.

Command:

```
tail -n +2 data/tickets.csv | cut -d, -f3 | sort | uniq -c | sort -nr
```

Plain English:

Skip the header, select category, sort categories,
count repeated categories, largest count first.

This is a strong exit-ticket prompt.

Where This Skill Shows Up

Give students a reason to care about old-looking commands.

Roles

- support engineer
- QA tester
- software developer
- cybersecurity analyst
- data analyst

Tasks

- search logs
- count failures
- inspect exports
- summarize incidents
- save evidence

If A Student Cannot Run Commands

They can still learn by reading, predicting, and explaining.

Have them do

- watch demo
- copy commands
- write predicted output
- explain each pipe stage
- answer reflection

Do not do

- spend 40 minutes installing tools
- debug one laptop forever
- make environment failure the lesson
- create grade panic

Ungraded Skill Practice

Students collect command evidence and writing for their own notes.

Evidence

- head output
- line count
- grep results
- category pipeline
- ticket_report.sh
- report.txt output

Writing

- what grep does
- what a pipe does
- most common category
- why find is useful
- job connection reflection

What Success Looks Like Today

Students can turn a text file into a basic report.

Students should be able to say

- I can search with grep
- I can explain a pipe
- I can count categories
- I can save output
- I can run a Bash script

Next class

- Python CSV reading
- SQLite database file
- tables and rows
- SQL queries
- persistence